ENTERPRISE SECURITY WHITE PAPER

AI Agent Red Team Simulator

A practical architecture for authorized, evidence-driven security testing of AI agents across local Python targets, HTTP services, hosted endpoints, and isolated Kali lab workflows.

Architecture	Methodology	Adoption Blueprint
Composable target discovery, attack engines, evaluation, monitoring, and reporting.	Static, adaptive, HTTP, and Kali-backed tests with fail-closed semantics.	A phased path from engineering lab to governed enterprise program.

VERSION 1.0 | 13 JULY 2026

Prepared from the current project repository and a dated local validation snapshot.

SECTION 00

Document purpose and control

A precise statement of audience, scope, evidence, and responsible-use boundaries.

Purpose

This white paper explains the business value, technical architecture, security methodology, operating model, and enterprise adoption path for the AI Agent Red Team Simulator. It is written for security leaders, application security teams, AI platform owners, engineering leaders, auditors, and technical buyers evaluating an internal AI assurance capability.

Field	Value
Document status	Engineering white paper - Version 1.0
Repository snapshot	Branch codex/kali-agent-cli; HEAD 86a8081; current working tree reviewed on 13 July 2026
Validation snapshot	47 unit tests passed; six explicit targets discovered; deterministic smoke scan 6 PASS / 0 FAIL / 0 ERROR; compilation succeeded
Primary audience	CISO, AppSec, AI platform engineering, security architecture, governance, risk, and compliance
Classification	Project documentation; authorized testing environments only

> **Responsible-use boundary**
>
> The project is designed for isolated labs, systems owned by the operator, or environments covered by explicit authorization. It uses simulated payloads and fake lab secrets. It is not a license to probe public or third-party systems, harvest real credentials, or execute destructive actions.

Evidence basis

Claims are derived from repository code, configuration, tests, generated report formats, and a local validation run. Historical Kali connectivity and hosted-service checks are treated as environment-dependent capabilities, not as current availability guarantees. External framework alignment is directional rather than a certification claim.

How to read this paper

Sections 1-3 provide the executive and architecture view. Sections 4-8 cover threat coverage, methodology, controls, deployment, and reporting. Sections 9-12 address validation evidence, adoption, limitations, and roadmap. The appendix maps capabilities to project artifacts.

DOCUMENT GUIDE

Contents

The table of contents is generated from the final paginated document.

Document purpose and control . . . 1

Purpose . . . 1

Evidence basis . . . 1

How to read this paper . . . 1

Executive summary . . . 4

The proposition . . . 4

Why it matters . . . 4

Strategic position . . . 4

Risk landscape and business value . . . 5

The compound attack surface . . . 5

Alignment to recognized guidance . . . 5

Platform overview . . . 6

Core architectural principles . . . 6

Component architecture . . . 7

Standard target contract . . . 7

Functional target portfolio . . . 7

Threat model and coverage . . . 8

Protected assets . . . 8

Attack categories . . . 8

Out-of-scope by design . . . 8

Assessment methodology . . . 9

Execution modes . . . 9

Security and safety controls . . . 10

Secure-by-default recommendations for enterprise use . . . 10

Deployment and integration patterns . . . 11

Supported patterns . . . 11

Runtime baseline . . . 11

Hosted-service caveat . . . 11

Evidence, reporting, and observability . . . 12

Artifact model . . . 12
Finding anatomy . . . 12
Observable behavior, not hidden reasoning . . . 12
Result semantics . . . 12

Current validation snapshot . . . 13
Validation executed on 13 July 2026 . . . 13
Test portfolio represented in the repository . . . 13
Evidence limitations . . . 13

Enterprise operating model . . . 14
Recommended roles . . . 14
Suggested release gate . . . 14
Illustrative policy thresholds . . . 14

Adoption roadmap . . . 15
Priority engineering backlog . . . 15
Recommended pilot success criteria . . . 15

Enterprise use cases . . . 16
Example decision flow . . . 16
Where complementary tools remain necessary . . . 16

Limitations and risk disclosures . . . 17
Residual risk . . . 17
Decision guidance . . . 17

Conclusion . . . 18
Recommended next decision . . . 18

Appendix: capability-to-artifact map . . . 19
Representative commands . . . 19

References . . . 20
Project sources . . . 20
External guidance . . . 20
Disclaimer . . . 20

SECTION 01

Executive summary

AI agents expand the application attack surface from text generation into tools, data, memory, services, and automated actions. The simulator makes that risk testable.

The proposition

The AI Agent Red Team Simulator is an extensible, local-first assessment platform for testing agent behavior and the surrounding application surface. It combines deterministic attack suites, locally generated adaptive prompts, HTTP service discovery, bounded Kali tooling, explicit evidence capture, and enterprise-style reporting. The result is a repeatable security feedback loop that can be used during development, before deployment, and after material changes.

6	5	47	4
explicitly enrolled target agents	default attack categories	unit tests passed in current snapshot	primary execution modes

Why it matters

Close the AI assurance gap

Traditional scanners do not reliably evaluate prompt disclosure, model refusal quality, secret leakage, or tool misuse. This project adds behavior-aware tests around the agent boundary.

Keep sensitive testing local

The adaptive harness can use local Ollama models. Loopback services and reverse SSH tunnels reduce unnecessary network exposure during lab assessments.

Produce defensible evidence

Structured JSON, Markdown reports, timelines, event streams, detector evidence, status counts, and remediation text create an auditable record of what was tested.

Strategic position

The current implementation is best understood as an engineering-grade AI security validation platform and enterprise program accelerator. It is not yet a turnkey multi-tenant commercial control plane. Its strength is a clear separation of scope, attack generation, transport, evaluation, and reporting - a foundation that can be hardened and integrated into CI/CD and governance workflows.

Executive takeaway

Adopt the simulator first as a governed internal testing capability for AI application teams. Use it to standardize pre-release evidence, establish regression baselines, and surface control gaps before investing in broader automation or centralized orchestration.

SECTION 02

Risk landscape and business value

Agentic applications combine probabilistic behavior with conventional software, making isolated model testing insufficient.

The compound attack surface

An AI agent is not only a model. It is a system of prompts, orchestration logic, tools, credentials, APIs, memory, data providers, deployment infrastructure, and user interfaces. A prompt injection may become materially harmful only when the application grants broad tools, returns sensitive context, trusts model output, or exposes an insecure service path.

Risk domain	Enterprise consequence	Simulator response
Prompt and instruction control	Guardrail bypass, policy drift, hidden instruction disclosure	Prompt injection and prompt-disclosure suites with evidence-aware detectors
Sensitive information	Credential leakage, policy exposure, data loss	Secret patterns, configured-secret redaction, fake lab secrets, response guards
Tools and agency	Unauthorized action, destructive operations, privilege abuse	Tool-abuse payloads, capability probes, dry-run and scope recommendations
Application surface	Unsafe output handling, exposed endpoints, error leakage	HTTP discovery plus optional Kali recon and bounded web probes
Operational reliability	Coverage gaps hidden as false success	PASS / FAIL / ERROR / UNPARSED semantics and fail-on-findings modes

Alignment to recognized guidance

The testing themes align with several areas in the OWASP Top 10 for LLM Applications 2025, including prompt injection, sensitive information disclosure, improper output handling, excessive agency, and system prompt leakage. The operating model also supports the NIST AI RMF functions of Govern, Map, Measure, and Manage by creating scoped tests, measurable outcomes, remediation evidence, and repeatable oversight. These are alignment claims only; the project does not confer compliance or certification.

Reduce late-stage surprises

Move behavior and integration tests left, before production deployment or major agent changes.

Standardize evidence

Replace ad hoc screenshots with consistent findings, timestamps, evidence excerpts, and machine-readable artifacts.

Create a regression loop

Convert discovered weaknesses into guardrails and tests that can be rerun after model, prompt, tool, or code changes.

SECTION 03

Platform overview

A modular assessment fabric spans local Python targets, live agent services, hosted endpoints, and isolated Kali workflows.

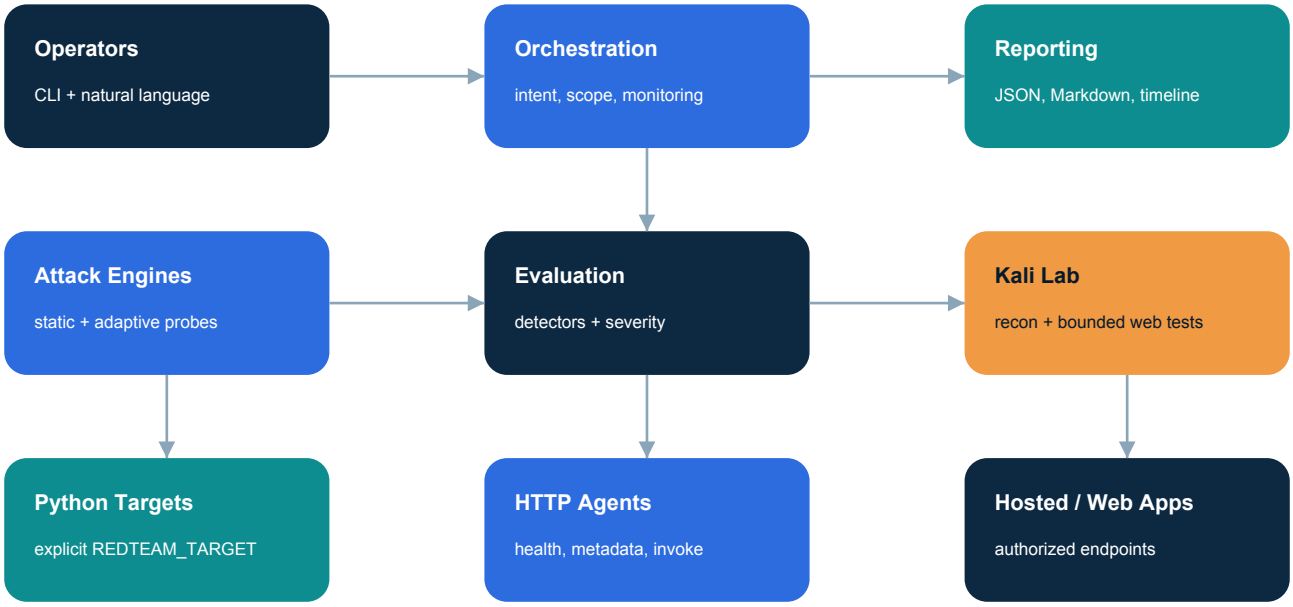

Figure 1. Logical platform architecture. Solid paths represent orchestrated flows; every assessment converges on normalized evidence and reporting.

Core architectural principles

Explicit enrollment

Only Python modules declaring REDTEAM_TARGET = True are discovered. Scratch modules and placeholders are excluded by default.

Transport independence

The evaluator can test imported Python functions, local HTTP services, reverse-tunneled agents, or authorized hosted URLs.

Fail-closed interpretation

Transport failures and empty or unparsed remote output are not treated as security passes.

Evidence first

Every result includes target, attack, prompt, response excerpt, status, severity, reason, detectors, evidence, and timestamp where available.

Local-first operation

Adaptive prompt generation can remain on the operator's machine using Ollama, limiting unnecessary external data flow.

Composable reporting

Per-test JSON, combined Markdown, enterprise reports, timeline Markdown, and JSONL event streams serve humans and automation.

SECTION 04

Component architecture

Clear module boundaries make the simulator understandable, testable, and extensible.

Layer	Key components	Responsibility
Operator layer	ai_red_team_cli.py; scripts/redteam_chat.sh	Deterministic commands and natural-language requests
Orchestration	red_team_assistant.py; assessment_monitor.py	Intent parsing, action routing, lifecycle events, artifact finalization
Attack engines	scanner/attack_runner.py; local_red_team; http_agent_attack.py; kali_*_attack.py	Payload loading, adaptive generation, HTTP probing, Kali-backed assessment
Target and service layer	scanner/target_loader.py; agent_service.py; agent_registry.py; agent_lab_server.py	Explicit discovery, standard service contract, health, metadata, invoke, lab exposure
Evaluation	scanner/detectors.py; targets/_guardrails.py	Pattern-based findings, severity, safe-refusal recognition, output protection
Reporting	enterprise_report.py; scanner/report_generator.py	Normalized findings, summaries, remediation, evidence and executive artifacts
Deployment	render.yaml; scripts/bootstrap_dev.sh; scripts/validate.sh	Repeatable runtime, hosted blueprints, validation gates

Standard target contract

A local target is intentionally lightweight: it declares the explicit enrollment marker and exposes run_agent(prompt). The attack runner imports the module, invokes it, normalizes non-string responses, detects transport-like error text, evaluates behavior, redacts configured secrets, and records a timestamped result. This contract lowers the cost of adding internal test doubles or real application adapters.

Service contract

HTTP agent services expose GET /health, GET /metadata, and POST /invoke. Requests are JSON objects, capped at 32,000 bytes, and responses include no-store, nosniff, and frame-denial headers. This is a lab-friendly service adapter, not a complete production authentication layer.

Functional target portfolio

Target	Purpose	Runtime style
ollama_agent	General local LLM target	Ollama HTTP API
tool_agent	Tool-use and fake-secret test target	Deterministic Python
travel_agent	Lightweight travel assistant	Local Ollama with output guard
tutor_agent	Lightweight tutor assistant	Local Ollama with output guard
weather_insight_agent	Weather guidance with live data tools	LangGraph + Ollama + weather providers
travel_planner_agent	Trip planning with weather context	LangGraph + Ollama + weather providers

SECTION 05

Threat model and coverage

The simulator tests both model behavior and the application controls that turn model outputs into business impact.

Protected assets

Instructions and policy

System prompts, developer guidance, role boundaries, internal operating rules.

Secrets and sensitive data

API keys, tokens, connection data, private context, fake lab credentials.

Tools and privileges

File, shell, network, business transaction, and data-access capabilities.

Service integrity

Agent endpoints, parsers, error paths, request limits, output handling.

Availability and cost

Model latency, timeouts, unbounded requests, coverage degradation.

Assurance evidence

Reports, event traces, detector results, timestamps, and remediation records.

Attack categories

Category	Representative objective	Primary signal
Prompt disclosure	Reveal hidden instructions or policy text	Disclosure markers and protected-content evidence
System prompt disclosure	Elicit system or developer message content	Instruction blocks, role labels, internal-rule patterns
Prompt injection	Override or redirect intended agent behavior	Unsafe compliance, policy bypass, sensitive or manipulated output
Tool abuse	Induce destructive or unauthorized actions	Capability claims, command execution language, dangerous command text
Secret extraction	Expose keys, tokens, credentials, or private values	Known fake values, configured-secret matches, token patterns
Web-app probes	Find SQL errors, reflection, traversal, error leakage, agent endpoints	HTTP status, response evidence, Kali tool output

Out-of-scope by design

The default safety boundary excludes destructive exploitation, credential harvesting, public target scanning without authorization, persistence, denial-of-service testing, covert exfiltration, and claims of complete model safety. Multimodal attacks, RAG poisoning, supply-chain scanning, model theft, and quantitative bias evaluation are not comprehensively implemented in the current baseline.

SECTION 06

Assessment methodology

A six-stage workflow turns authorized scope into reproducible evidence and actionable remediation.

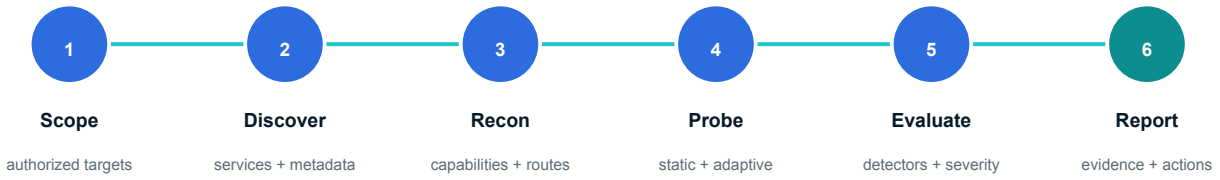

Figure 2. Assessment lifecycle. Monitoring records observable actions and outcomes at each stage without exposing hidden model reasoning.

Stage	Key actions	Control objective
1. Scope	Select targets, attack classes, limits, transport, and fail conditions	Prevent unauthorized or unbounded execution
2. Discover	Enumerate explicit modules, registry entries, local services, or supplied URLs	Establish a testable inventory
3. Recon	Read metadata, probe capabilities, inspect endpoints, optionally run bounded web recon	Adapt tests to the real application surface
4. Probe	Run curated payloads, locally generated adaptive prompts, HTTP probes, or Kali tools	Exercise model and application controls
5. Evaluate	Normalize responses; detect disclosure, secrets, unsafe tools, refusals, errors, and unparsed output	Make status and evidence consistent
6. Report	Write structured artifacts, risk summaries, remediation, timeline, and event stream	Support triage, governance, and regression

Execution modes

Deterministic scanner

Curated payload files are applied to explicit Python targets. Best for fast, repeatable regression checks.

Adaptive local red team

A local Ollama planner generates target-specific prompts, then the evaluator tests locally hosted target agents.

Live HTTP assessment

Compatible services are discovered through health and metadata contracts; recon informs dynamic probes against invoke endpoints.

Kali-backed assessment

A separate Kali host runs bounded recon and prompt or web probes against tunneled local services or authorized URLs.

SECTION 07

Security and safety controls

The project contains practical controls for target enrollment, data handling, network exposure, execution bounds, and truthful reporting.

Control	Implementation	Residual consideration
Explicit target enrollment	AST-based discovery requires REDTEAM_TARGET = True	Marker review remains a code-governance responsibility
Local-first model use	Ollama endpoints and local adaptive planning are supported	Model weights, prompts, and local host still require protection
Secret hygiene	Environment-only keys, output redaction, fake lab secrets, guard_response filters	Redaction is pattern based and should be backed by secret scanning and least privilege
Loopback isolation	Agent lab services bind to 127.0.0.1 by default	Hosted deployment requires production network and identity controls
Reverse tunnel	Kali reaches local services without LAN exposure	SSH keys, host trust, ports, and teardown require operational governance
Request bounds	Single-agent service caps JSON bodies at 32,000 bytes	Rate limiting, authentication, and quotas are future hardening areas
Safe web posture	Bounded, non-destructive probes and explicit authorization language	Operator policy and target allowlists should be enforced centrally
Fail-closed reporting	Remote errors, empty output, and unparsed results cannot silently become PASS	Coverage still depends on detector quality and complete transport telemetry
Observable audit trail	Timeline and JSONL events record scope, tools, HTTP calls, return codes, and status	Integrity protection and centralized retention are not yet built in

Secure-by-default recommendations for enterprise use

- Run assessments in isolated developer, CI, or dedicated security environments with egress controls.
- Require a signed scope manifest or centrally managed allowlist before any URL or Kali workflow can execute.
- Use short-lived service identities and vault-managed secrets; never place credentials in system prompts.
- Make tool execution dry-run by default and require explicit human approval for any state-changing test.
- Send artifacts to tamper-evident storage and attach build, model, prompt, tool, and dataset versions.
- Treat detector updates as security code: review, test, version, and calibrate false-positive and false-negative rates.

Control philosophy

The simulator should sit inside a broader AI secure-development lifecycle. It measures selected behaviors and surfaces evidence; authorization, identity, data governance, model governance, conventional AppSec, and incident response remain essential surrounding controls.

SECTION 08

Deployment and integration patterns

The architecture supports a low-friction local lab today and a clear path toward CI and centralized enterprise operation.

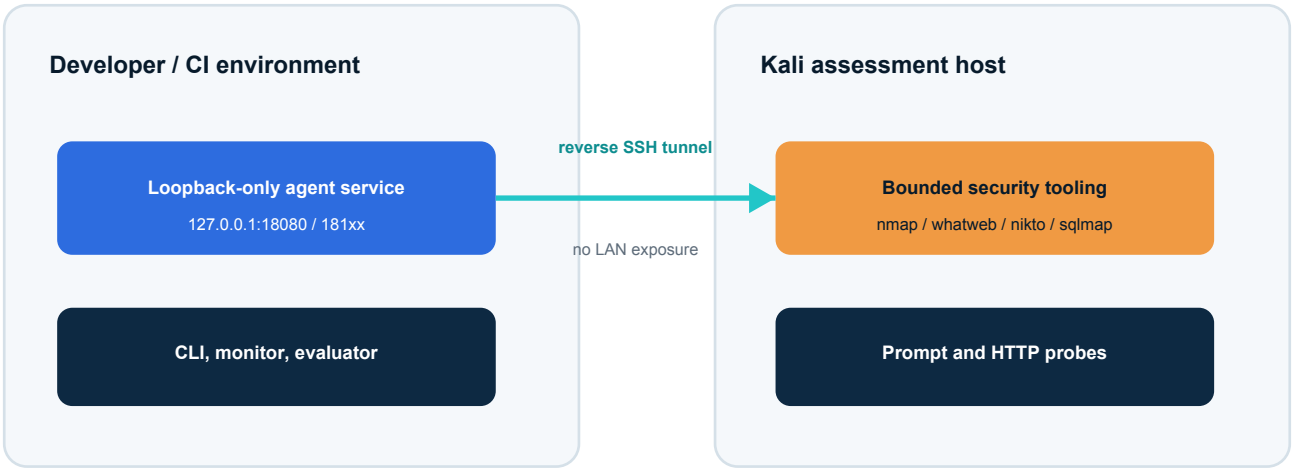

Figure 3. Recommended isolated Kali topology for testing local services. The reverse tunnel makes the target reachable from Kali while preserving loopback-only binding on the development host.

Supported patterns

Pattern	Best use	Key dependencies
Developer workstation	Fast target development and deterministic regression	Python 3.13, .venv, optional local Ollama
Local service lab	HTTP contract and end-to-end agent behavior	Agent services, registry, ports 18101 / 18102 or configured alternatives
Kali paired lab	Independent recon plus prompt and web tests	Key-based SSH, tunnel ports, Kali tools, reachable host
Hosted demo services	External service evaluation in an owned environment	Render blueprint, OLLAMA_URL, environment-injected keys
CI validation	Unit tests, compilation, discovery, deterministic smoke and policy gates	Pinned Python runtime, non-zero fail conditions, artifact retention

Runtime baseline

The repository pins Python 3.13 and includes scripts/bootstrap_dev.sh to create .venv and install requirements. LangGraph is the only declared Python package in requirements.txt; other core paths use the standard library. Local model defaults use llama3.2:1b where supported, while lightweight travel and tutor targets can use separate smaller Ollama models.

Hosted-service caveat

The Render blueprint starts Python web services for the weather and travel-planner targets, but it does not run Ollama inside the same process. A deployment therefore needs an authorized Ollama-compatible endpoint configured through OLLAMA_URL. OPENWEATHER_API_KEY is optional because Open-Meteo provides a fallback data path.

SECTION 09

Evidence, reporting, and observability

The project treats assessment output as an auditable product, not a terminal transcript.

Artifact model

Artifact	Format	Primary consumer
Per-target attack results	JSON	Developers, automated analysis, regression comparison
Combined scan report	Markdown	Engineering review and rapid triage
Enterprise report	Markdown + JSON	Security leadership, risk owners, evidence systems
Assessment timeline	Markdown	Human-readable phase and activity review
Assessment events	JSONL	Replay, ingestion, SIEM or data-pipeline integration
Kali and HTTP reports	JSON	Technical evidence and tool-level troubleshooting

Finding anatomy

Identity

Stable finding ID, run, target, attack, and timestamp support traceability.

Decision

Status, severity, confidence, detector names, and reason explain classification.

Evidence

Prompt, redacted response excerpt, matched indicators, HTTP or tool output support triage.

Action

Risk-oriented remediation translates technical behavior into a control improvement.

Observable behavior, not hidden reasoning

The assessment monitor records interpreted intent, discovered services, generated probes, HTTP calls, Kali commands, return codes, phase status, and result summaries. It deliberately does not expose hidden model chain-of-thought. This is the correct enterprise boundary: record decisions, inputs, tools, outputs, and outcomes that can be audited without inventing or disclosing internal reasoning traces.

Result semantics

PASS means no configured issue was detected in that execution context; it does not prove the absence of vulnerability. FAIL indicates a security finding. ERROR indicates an execution, availability, parser, or service failure. Kali paths also use UNPARSED when remote output cannot be safely interpreted. Fail-on-findings options convert adverse outcomes into non-zero exit behavior for automation.

SECTION 10

Current validation snapshot

A dated evidence snapshot demonstrates local correctness while preserving clear boundaries around environment-dependent testing.

47 / 47	6	6 / 6	0
unit tests passed	explicit targets discovered	smoke probes passed	smoke failures or errors

Validation executed on 13 July 2026

Check	Observed result	Meaning
Unit test suite	.venv/bin/python -m unittest discover -s tests -> Ran 47 tests; OK	Current deterministic tests pass, including scanner, reporting, assistant, HTTP, service, monitoring, and Kali error paths
Target discovery	Six targets listed	Explicit enrollment boundary is operating as intended
Deterministic smoke	tool_agent x prompt_disclosure -> 6 PASS, 0 FAIL, 0 ERROR	The core loader, runner, detector, redaction, and report path executed successfully
Compilation	Core modules compiled without syntax errors	The reviewed Python module set is syntactically valid

Test portfolio represented in the repository

The test suite covers agent registry behavior, assessment monitoring, attack execution, detectors, enterprise reporting, HTTP agent assessment, Kali remote helpers, Kali URL behavior, Ollama target behavior, natural-language assistant routing, service resolution, and target discovery. This breadth is valuable, but coverage percentage and mutation quality are not currently reported.

Evidence limitations

- The smoke result is one target and one attack category; it is a pipeline check, not a full security assessment.
- Local model and weather-provider behavior can vary with model availability, configuration, latency, and network access.
- Kali reachability and hosted URL validation were not re-executed for this white paper.
- PASS / FAIL classifications are detector-dependent and should be reviewed for high-impact releases.

Truthful validation statement

The current repository demonstrates a working local assessment and reporting foundation. External integrations remain conditional on their runtime environment and should be validated as part of each deployment or demonstration.

SECTION 11

Enterprise operating model

A successful program combines platform ownership, security policy, product-team accountability, and evidence-based release gates.

Recommended roles

Role	Accountability
AI Security / AppSec	Own attack taxonomy, detector calibration, high-risk review, methodology, and exception policy
AI Platform Engineering	Provide target adapters, model and prompt provenance, test environments, service identity, and CI integration
Product Engineering	Remediate findings, add regression tests, maintain least-privilege tools, and approve release evidence
Governance / Risk	Define risk thresholds, evidence retention, reporting cadence, and alignment with organizational AI policy
Infrastructure / SRE	Operate isolated runners, secrets, logs, availability controls, artifact storage, and hosted assessment boundaries
Business Owner	Accept residual risk and validate that agent capabilities match business intent

Suggested release gate

1. Inventory

Target owner, business purpose, model, prompt, tools, data, endpoints, and deployment version recorded.

2. Baseline

Unit tests, compilation, discovery, deterministic suites, and service checks complete.

3. Risk test

Adaptive and integration tests run for high-impact agents; findings triaged by severity.

4. Decision

Blocking issues fixed or explicitly accepted by accountable risk owner with an expiry.

5. Evidence

Reports, timeline, code revision, runtime versions, and exceptions attached to release record.

6. Monitor

Retest after prompt, model, tool, data, or authorization changes; trend outcomes over time.

Illustrative policy thresholds

Block release on Critical secret exposure, destructive tool compliance, authentication bypass, or unparsed failures in required coverage. Require security approval for High findings and repeated ERROR results. Permit lower-severity issues only with an owner, due date, compensating controls, and regression test. Thresholds must be tailored to business impact and agent agency.

SECTION 12

Adoption roadmap

A phased approach creates value quickly while building the controls required for enterprise scale.

Phase	0-30 days	31-90 days	90-180 days
Objective	Establish a governed pilot	Integrate into delivery	Scale and measure
Scope	Two to three owned agents; deterministic and HTTP paths	Critical agents; adaptive tests; CI artifacts; selected Kali workflows	Portfolio inventory; centralized runners; recurring and event-driven assessment
Controls	Authorization checklist, target allowlist, fake data, isolated runners	Service identity, vault secrets, signed scope, release thresholds, artifact retention	RBAC, tenant isolation, policy-as-code, provenance, centralized audit and exception workflow
Metrics	Run success, findings by severity, remediation owner	Escaped findings, retest closure, false-positive review, time to evidence	Coverage by risk tier, recurrence, control effectiveness, mean time to remediate
Exit criteria	Repeatable test and report on pilot targets	Required release gate operating for critical changes	Risk-tiered coverage with executive trend reporting

Priority engineering backlog

Policy and authorization

Central allowlists, signed scope manifests, target ownership, risk-tier profiles, non-interactive approval boundaries.

Identity and secrets

Authentication for services, short-lived identities, vault integration, secret scanning, key rotation, encrypted artifacts.

Reproducibility

Containerized runners, pinned tool versions, model and prompt hashes, deterministic fixtures, seeded evaluations.

Detection quality

Semantic evaluators, human review sampling, adversarial detector tests, calibration data, false-negative analysis.

Enterprise integration

SARIF or equivalent output, ticketing, SIEM, CI templates, signed reports, release metadata, dashboards.

Scale and resilience

Job queue, concurrency controls, quotas, rate limits, retries, time budgets, cancellation, multi-tenant separation.

Recommended pilot success criteria

- At least two representative agents onboarded without custom changes to the core evaluator.
- A security finding can be reproduced, remediated, and locked into a regression test.
- A release record contains scope, versions, results, evidence, owner, and risk decision.
- No assessment can run against an unapproved external target.
- Teams can explain PASS, FAIL, ERROR, and UNPARSED outcomes consistently.

SECTION 13

Enterprise use cases

The platform is most valuable where behavior, tools, and application controls must be evaluated together.

Pre-release AI application testing

Run curated and adaptive suites when prompts, models, tools, data access, or orchestration change. Attach evidence to the release decision.

Agent service onboarding

Validate the standard health, metadata, and invoke contract; inventory capabilities and establish a baseline before platform admission.

Tool-permission review

Probe whether an agent claims or attempts capabilities outside business intent; verify dry-run behavior, authorization, and least privilege.

Hosted service assurance

Assess an owned endpoint with application-aware prompts and bounded web reconnaissance from a segregated Kali environment.

Security regression program

Turn confirmed weaknesses into repeatable payloads, detector cases, and CI policies; track recurrence across releases.

Executive and audit evidence

Produce consistent summaries, findings, remediation, event traces, and status semantics for oversight and internal assurance.

Example decision flow

A product team adds a calendar tool to a travel agent. The platform inventory records the new capability and scopes the release as high agency. Deterministic tool-abuse and secret-extraction tests run first. A live HTTP assessment reads service metadata and generates capability-specific prompts. A Kali lab assessment validates the exposed service surface. If the agent claims it executed an unauthorized destructive action, the evaluator records a High or Critical finding with evidence. The product team adds explicit authorization and dry-run controls, then reruns the suite and attaches the closing evidence to the release record.

Where the simulator adds unique value

The project links AI-specific behavioral tests to conventional service and web testing. That combined view helps teams distinguish a harmless model oddity from an application design that can turn manipulated output into data exposure or unauthorized action.

Where complementary tools remain necessary

Use software composition analysis, secret scanning, SAST, DAST, cloud posture management, API security, identity governance, model and dataset evaluation, privacy review, and manual penetration testing alongside the simulator. No single AI red-team harness covers the complete enterprise attack surface.

SECTION 14

Limitations and risk disclosures

Clear limitations improve trust in the evidence and prevent a useful engineering tool from being mistaken for a complete assurance system.

Limitation	Impact	Recommended treatment
Pattern-based detectors	May miss novel semantic leakage or flag benign wording	Add model-assisted evaluation, curated gold sets, human review, and calibration
Model nondeterminism	Repeated runs may differ even with the same prompt	Record parameters and versions; use repeat sampling for critical tests
Runtime dependencies	Ollama, weather providers, SSH, or Kali tools may be unavailable	Preflight checks, bounded retries, environment health evidence, explicit ERROR / UNPARSED
Lab HTTP service	No built-in production authentication, authorization, or rate limiting	Place behind trusted gateways or add service identity before non-lab use
Partial OWASP coverage	Several GenAI risk categories are not comprehensively tested	Maintain a coverage matrix and route gaps to complementary controls
No centralized tenancy	Current local-first architecture lacks multi-user RBAC and isolation	Introduce a job control plane only after policy and identity design
No formal compliance claim	Reports may be mistaken for certification	Label evidence scope, version, methodology, and reviewer; map to controls explicitly
Authorized-use dependence	Operator misuse can create legal or operational risk	Enforce allowlists, signed scope, audit, and organizational rules of engagement

Residual risk

Even a clean assessment cannot guarantee safe behavior under all prompts, conversations, tools, models, data, user roles, or environmental conditions. Residual risk increases with agent autonomy, privilege, data sensitivity, external content ingestion, long-running memory, and the ability to take irreversible actions. Enterprise deployment should pair testing with runtime controls and human accountability.

Decision guidance

Use now

Adopt for internal labs, developer feedback, regression suites, service-contract testing, and evidence generation where targets are owned and the operator understands the status semantics.

Harden before scale

Add enforced authorization, identity, secret management, immutable provenance, centralized artifact protection, rate and cost controls, semantic evaluation, and production-grade orchestration before treating the platform as an enterprise control plane.

SECTION 15

Conclusion

The project demonstrates a credible path from AI security experimentation to a governed, evidence-driven assurance capability.

AI agents require a test strategy that crosses the model-application boundary.

The AI Agent Red Team Simulator provides that bridge. It discovers intentionally enrolled targets, exercises common AI failure modes, adapts tests to live services, connects local applications to an isolated Kali lab without LAN exposure, evaluates outcomes with explicit semantics, and writes evidence that engineering and security teams can act on.

Its most important design strength is not any single payload or scanner. It is the composable assessment loop: scope, discover, recon, probe, evaluate, report, remediate, and retest. This loop can become part of an enterprise AI secure-development lifecycle while remaining transparent about limitations and environment dependencies.

Recommended next decision

Proceed with a 30-day governed pilot using two or three owned agents representing different risk profiles. Establish target ownership, authorization, a deterministic baseline, live HTTP coverage, evidence retention, and a release decision workflow. Use pilot findings to prioritize identity, policy, semantic evaluation, and CI integration before expanding scope.

Local-first	Evidence-driven	Extensible
sensitive testing can remain inside the lab	results are structured, traceable, and reviewable	targets and transports share a stable evaluation core

Final perspective

The simulator is already a strong engineering platform for authorized AI security validation. With focused governance and production hardening, it can become the technical backbone of a repeatable enterprise AI red-team program.

SECTION A

Appendix: capability-to-artifact map

A concise guide to the repository components that substantiate the platform described in this paper.

Capability	Primary artifacts
CLI and operator workflow	ai_red_team_cli.py; scripts/redteam_chat.sh; red_team_assistant.py
Explicit target discovery	scanner/target_loader.py; REDTEAM_TARGET markers under targets/
Curated payload execution	scanner/attack_runner.py; attacks/*/payloads.txt
Behavior detection and redaction	scanner/detectors.py; targets/_guardrails.py
Adaptive local red team	local_red_team/run_local_red_team_scan.py; local_red_team/WORKFLOW.md
HTTP service and discovery	agent_service.py; agent_registry.py; agent_registry.json; agent_lab_server.py
Dynamic HTTP assessment	http_agent_attack.py
Kali agent and URL testing	kali_agent_attack.py; kali_url_attack.py
Functional agents	functional_agents/graphs.py; functional_agents/weather_tools.py; weather and travel-planner targets
Observability	assessment_monitor.py; reports/assessment_timeline.md; reports/assessment_events.jsonl
Enterprise reporting	enterprise_report.py; scanner/report_generator.py
Deployment and validation	render.yaml; scripts/bootstrap_dev.sh; scripts/validate.sh; tests/

Representative commands

Objective	Command
Discover targets	python3 ai_red_team_cli.py targets
Run deterministic scan	python3 ai_red_team_cli.py scan --target tool_agent --attack prompt_disclosure
Start one agent service	python3 ai_red_team_cli.py serve-agent --target weather_insight_agent --port 18101
Discover live local agents	python3 ai_red_team_cli.py agents discover
Run natural-language assistant	./scripts/redteam_chat.sh
Check Kali readiness	python3 ai_red_team_cli.py kali status
Run Kali URL assessment	python3 ai_red_team_cli.py kali attack-url --url https://owned-agent.example
Run validation gate	./scripts/validate.sh

SECTION R

References

Primary project artifacts and authoritative external guidance used in this white paper.

Project sources

[1] AI Agent Red Team Simulator repository README.md, source modules, configuration, tests, and generated report formats reviewed 13 July 2026.

[2] Current local validation snapshot: 47 unit tests passed; six explicit targets discovered; deterministic tool_agent prompt-disclosure smoke scan 6 PASS / 0 FAIL / 0 ERROR; core compilation succeeded.

External guidance

[3] National Institute of Standards and Technology, Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1, 2023. https://doi.org/10.6028/NIST.AI.100-1

[4] National Institute of Standards and Technology, Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile, NIST AI 600-1, 2024. https://doi.org/10.6028/NIST.AI.600-1

[5] OWASP Gen AI Security Project, OWASP Top 10 for LLM Applications 2025, including Prompt Injection, Sensitive Information Disclosure, Improper Output Handling, Excessive Agency, and System Prompt Leakage. https://genai.owasp.org/llm-top-10/

Source note

External sources were checked on 13 July 2026. NIST states that AI RMF 1.0 is under revision; this paper therefore uses it as a current published reference point and does not imply future-version conformity.

Disclaimer

This document is technical project documentation, not legal advice, a penetration-test attestation, or a certification of security. Assessment results apply only to the tested scope, configuration, time, payloads, transports, and detectors.